

JS10041003111~

שאלה 1 (35 נקודות)

שאלה זו עוסקת בשדרוג של שפת "וישגור" (שפת PROC) המתוארת בספר הלימוד בפרק 3 בעמודים 74-81.

בשאלה זו נרצה להוסיף לשפה תמיכה בביצוע לולאת foreach המאפשרת לבצע פעולה על סדרת איברים.

להלן הדקדוק המגדיר את לולאת foreach :

$Expression ::= \text{foreach } (Type\ identifier\ in\ [\{ Expression \}^{(*)}])$
 $\text{do } Expression$

$\text{foreach-exp } (typ\ id\ exps\ body)$

$Type ::= int \mid bool \mid func$

הסבר על המרכיבים של הביטוי החדש:

לביטוי foreach-exp המרכיבים הבאים :

- **typ** – הוא אחד מהמילים המוגדרות ע"י Type, ומציין את טיפוס הערכים במרכיב exps בהם בלבד יש להשתמש.
המילה int מציינת שמדובר בלולאה המבוצעת רק על ביטויים שערכם מספר שלם מתוך כלל הביטויים ברשימה exps.
המילה bool מציינת שמדובר בלולאה המבוצעת רק על ביטויים שערכם בוליאני מתוך כלל הביטויים ברשימה exps.
המילה func מציינת שמדובר בלולאה המבוצעת רק על ביטויים שערכם הוא פרוצדורה מתוך כלל הביטויים ברשימה exps.
- **id** – שם מזהה חדש המוכר רק במסגרת ביצוע הלולאה, ובכל מחזור לולאה תבוצע לו כריכה (binding) לערך הבא המתאים מ-exps.
- **exps** – רשימת ביטויים כלשהם (מסוגים שונים) שאת ערכם יש לסקור במסגרת הלולאה. אלה הם הערכים אליהם ייכרך id בכל מחזור לולאה (בכל פעם לערך הבא מהסוג שהוצהר)
- **body** - ביטוי המתאר את גוף הלולאה שיש לבצע.

ביטוי foreach מחזיר את כמות הערכים מתוך הרשימה exps עבורם בוצע גוף הלולאה.

דוגמה להמחשה:

דוגמה :

```
> (run
  "
    foreach (int i in [10,20,zero?(8), proc(x) zero?(x) ])
    do
      - (i,2)
    ")
(num-val 2)
>
```

הסבר הדוגמה:

בדוגמה מוצגת לולאת foreach הפועלת על רשימת ביטויים הכוללת 2 ביטויים שערכם מספר, ביטוי אחד בוליאני, וביטוי אחרון שהוא פרוצדורה. הלולאה מבצעת את גוף הלולאה רק עבור הביטויים שערכם הוא מספר (כי זה הטיפוס אליו היא הוגדרה). ולכן הערך המוחזר של כל ביטוי ה-foreach הוא 2.

שימו לב, חשוב לבצע את גוף הלולאה עבור הביטויים המתאימים למרות שאין שימוש בערכם המוחזר.

ממשו והטמיעו את השינויים הדרושים בתוך שפת "וישגור" (שפת PROC).

הקפידו להסביר היכן בדיוק נדרשים שינויים ותוספות בקבצי המפרש, ומהם השינויים והתוספות.

הנחיות כלליות לפתרון:

1. הקפידו על כתב ברור, ועל קוד מבני ומסודר.
2. הקפידו על פתיחת וסגירת סוגריים במקומות הנכונים
3. הפתרון אמור לשמור על הגדרות השפה המקוריות (פרט למקרים בהם נדרש שינוי כזה במפורש בשאלה)
4. על מנת לפשט קוד ארוך, כדאי ורצוי להיעזר בכתיבת פרוצדורות עזר (מחוץ ל-value-of או כאלה המוטמעות בתוכו)
5. בפתרונכם, הקפידו על אבחנה ושימוש נכון בין ביטוי (expression) לבין תוצאת חישוב ביטוי (expval) לבין identifier
6. ניקוד יורד על אי ניתוח ומימוש נכון של הדקדוק הנתון בשאלה, כלומר יש להפקיד על זיהוי וטיפול נכון ב-terminals, non-terminals ופעולות של סגור (} *) או { }+ .

המשך הבחינה בעמוד הבא

שאלה 2 (40 נקודות)

בספר הלימוד בעמודים 113-120 מתוארת שפת "וירמוז"(IMPLICIT-REFS).

ברצוננו להרחיב שפה זו עם יכולות חדשות שיאפשרו הגדרה של generator ושימוש בו, בדומה לקיים בשפת פייתון.

נסביר תחילה מהו generator?

Generator הוא אוסף של ערכים, בהם ניתן להשתמש פעם אחת בלבד. לאחר שימוש בערכי ה-generator, הוא מתרוקן, ובשלב זה, לא ניתן לחלץ עוד ערכים ממנו.

להלן הדקדוק המגדיר את הביטויים החדשים שיש להטמיע בשפה:

Expression ::=

generator (Identifier) : [{ Expression }] yield Expression*

gen-exp (var exps retexp)

Expression ::= ::Identifier

return-exp (gen)

Expression ::= ??Identifier

empty-exp (gen)

הסבר על מרכיבי הביטויים ואופן פעולתם:

ביטוי gen-exp מורכב מהמרכיבים הבאים:

- var – שם של משתנה המוכר רק במסגרת פעולת ה-generator ובכל פניה ל-generator הוא יקבל את הערך הבא מרשימת הערכים שעוד נותרו ב-exps
 - exps – רשימת ביטויים שאת ערכן (אחד בכל פעם) יש לקשור ל-var בכל פעם שפונים ל-generator על מנת לחלץ את האיבר הבא.
 - retexp – ביטוי המתאר את הערך שיש להחזיר מה-generator בכל פעם שפונים אליו. הביטוי בונה ומחזיר את ה-generator. הביטויים השונים המוזכרים בהגדרת ה-generator יחושבו על פי הסביבה בעת הגדרת ה-generator (כלומר static-binding).
- שימו לב,** שיש כאן טיפוס נתונים חדש לשפה.

ביטוי **return-exp** מורכב מהמרכיבים הבאים :

- **gen** – הוא שם מזהה (identifier) של generator
הביטוי מחזיר את הערך הבא מ-gen במידה וקיים, אם לא נותרו ערכים תודפס הודעת שגיאה מתאימה.
במידה ו-gen אינו generator, תודפס הודעת שגיאה מתאימה.

ביטוי **empty-exp** מורכב מהמרכיבים הבאים :

- **gen** – הוא שם מזהה (identifier) של generator
הביטוי בודק האם gen הוא generator במצב ריק, ומחזיר תשובה בוליאנית בהתאם.
במידה ו-gen אינו generator, תודפס הודעת שגיאה מתאימה.

להלן דוגמאות להמחשת השימוש:

בכל הדוגמאות ניתן להניח שהשפה שודרגה עם פעולת כפל, בדומה להגדרה ומימוש של פעולת חיסור הקיימת בהגדרות השפה.

דוגמה 1:

```
> (run "
      let g= generator(x): [5 2 ] yield *(x,x)
      in
        - (::g, ::g) ")
(num-val 21)
>
```

דוגמה 1, מוגדר generator בשם g, המורכב מ-2 ערכים, ומחזיר את ריבועי ערכיו. במסגרת ביטוי החיסור מוחזר בתחילה 25 (5 בריבוע) ולאחר מכן, 4 (2 בריבוע), והתוצאה היא ההפרש ביניהם שהוא 21 (על פי 4-25).
הערה: לאחר החזרת הערך 4 מ-g, ה-generator התרוקן.

(המשך השאלה בעמוד הבא)

דוגמה 2:

```
> (run "
      let g= generator(x): [5 ] yield *(x,x)
      in
        - (::g, ::g) ")
⊗ ⊗ interp.scm:123:41: Generator is empty
```

דוגמה 2, מוגדר generator בשם g, המורכב מערך יחיד, ומחזיר את ריבועו. במסגרת ביטוי החיסור מוחזר בתחילה 25 (5 בריבוע), בשלב זה g התרוקן, ולכן ניסיון לחלץ שוב איבר מ-g מחזיר שגיאה.

דוגמה 3:

```
> (run "let i=10
      in
        let g= generator(x): [5 ] yield *(x,x)
        in
          - (::i, ::g) ")
⊗ ⊗ interp.scm:132:30: Gen-extract: i Not a generator
```

דוגמה 3, בשלב פעולת החיסור, מנסים תחילה לחלץ מ-i ערך, אך i אינו generator ולכן התקבלה שגיאה מתאימה על כך.

דוגמה 4:

```
> (run "
      let sum=0
      in
        letrec f(g)= if ??g then sum
                      else begin
                        set sum= -(sum, -(0, ::g));
                        (f g)
                      end
        in
          (f generator(x): [5 3] yield *(x,x))")
(num-val 34)
> |
```

דוגמה 4, מוגדר משתנה sum, לאחריו מוגדרת פרוצדורה רקורסיבית בשם f המקבלת פרמטר בשם g מסוג generator, ומחזירה את סכום ערכיו. בדוגמה זו, f מופעלת על ה-generator המכיל את הערכים [5 3] ומחזיר את ריבועיהם. ולכן, הוחזר 34 שהוא 5^2+3^2 .

שימו לב לשימוש בכל הביטויים החדשים ולאופן השימוש בהם כפי שהוגדר בשאלה.

(המשך השאלה בעמוד הבא)

ממשו והטמיעו את הביטויים החדשים בתוך שפת **"ויפנה"** (שפת EXPLICIT-REFS).
הקפידו להסביר **היכן בדיוק** נדרשים שינויים ותוספות בקבצי המפרש, **ומהם** השינויים והתוספות.

שימו לב, יש להקפיד על עבודה עם חוקי וכללי השפה הנתונה בשאלה (שפת "וירמוז" – Implicit-Refs)

יש לשים לב ולבדוק מקרים של שימוש לא תקין בתחביר.
במקרים של שימוש לא תקין, יש להדפיס הודעות שגיאה מתאימות כפי שהודגם.

הנחיות כלליות לפתרון:

1. הקפידו על כתב ברור, ועל קוד מבני ומסודר.
2. הקפידו על פתיחת וסגירת סוגריים במקומות הנכונים
3. הפתרון אמור לשמור על הגדרות השפה המקוריות. (כגון אופן ניהול הסביבה וסוגי הערכים בה, אופן ניהול ההחסן והערכים שניתן לשמור בו, וכדומה)
4. על מנת לפשט קוד ארוך, כדאי ורצוי להיעזר בכתיבת פרוצדורות עזר (מחוץ ל-value-of או כאלה המוטמעות בתוכו)
5. בפתרונכם, הקפידו על אבחנה ושימוש נכון בין ביטוי (expression) לבין תוצאת חישוב ביטוי (expval) לבין identifier
6. ניקוד יורד על אי ניתוח ומימוש נכון של הדקדוק הנתון בשאלה, כלומר יש להפקיד על זיהוי וטיפול נכון ב-terminals, non-terminals ופעולות של סגור (} *) או { }

המשך הבחינה בעמוד הבא

שאלה 3 (25 נקודות)

להלן נתונה תוכנית בשפת "ויקיש" (INFERRED):

```
let Cola=0
in
  let Fanta=1
  in
    (proc (a: ?) zero?(if a then Cola else Fanta) zero?(Cola))
```

ברצוננו לקבוע מהו טיפוס התוכנית (אם קיים). לשם כך, עליכם להשתמש באלגוריתם שנלמד בפרק 7 להקשת הטיפוס.

- א. הקצו לביטוי הנתון בשאלה ולתתי הביטויים שלו משתני-טיפוס (Type Variables) מתאימים, וחברו משוואות מתאימות לביטוי הנתון ולתתי הביטויים שלו.
- ב. פתרו את המשוואות שהרכבתם בסעיף א' תוך שימוש ב-substitution ו-unification בדומה למתואר בספר בעמודים 252-258. בסיום פתרון המשוואות רשמו מהו הטיפוס שאותו הקשתם עבור התוכנית הנתונה.

בהצלחה !